



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference

*Wenjie Qu, National University of Singapore; Yijun Sun, Hong Kong University
of Science and Technology; Xuanming Liu, Tao Lu, and Yanpei Guo,
National University of Singapore; Kai Chen, Hong Kong University of Science
and Technology; Jiaheng Zhang, National University of Singapore*

<https://www.usenix.org/conference/usenixsecurity25/presentation/qu-zkgpt>

**This paper is included in the Proceedings of the
34th USENIX Security Symposium.**

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the
34th USENIX Security Symposium is sponsored by USENIX.

zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference

Wenjie Qu^{1,*}, Yijun Sun^{2,*}, Xuanming Liu¹, Tao Lu¹,
Yanpei Guo¹, Kai Chen², Jiaheng Zhang^{1,†}

¹National University of Singapore, ²Hong Kong University of Science and Technology

Abstract

Large Language Models (LLMs) are widely employed for their ability to generate human-like text. However, service providers may deploy smaller models to reduce costs, potentially deceiving users. Zero-Knowledge Proofs (ZKPs) offer a solution by allowing providers to prove LLM inference without compromising the privacy of model parameters. Existing solutions either do not support LLM architectures or suffer from significant inefficiency and tremendous overhead. To address this issue, this paper introduces several new techniques. We propose new methods to efficiently prove linear and non-linear layers in LLMs, reducing computation overhead by orders of magnitude. To further enhance efficiency, we propose constraint fusion to reduce the overhead of proving non-linear layers and circuit squeeze to improve parallelism. We implement our efficient protocol, specifically tailored for popular LLM architectures like GPT-2, and deploy optimizations to enhance performance. Experiments show that our scheme can prove GPT-2 inference in less than 25 seconds. Compared with state-of-the-art systems such as Hao et al. (USENIX Security'24) and ZKML (Eurosys'24), our work achieves nearly $279\times$ and $185\times$ speedup, respectively.

1 Introduction

Recently, Large Language Models (LLMs) (e.g., GPT-4 [2]) have revolutionized dialogue synthesis, applied in ChatBots [3], web search [33], and assisting programmers [63]. Due to their demanding resource requirements, AI companies typically deploy LLMs on cloud servers. Users access these models through APIs. Considering the huge daily serving costs, service providers are incentivized to cheat by using smaller models and giving lower-quality results to save costs. In fact, many users have observed the performance degradation of GPT-4 and ChatGPT [21, 22, 51] and suspect that OpenAI is using a smaller model. However, the inference

process is black-boxed to users. The service providers will not reveal the model for users to validate the service since the model parameters are trade secrets [10]. Hence, it is difficult for users to validate the service's integrity, e.g., the specified model computes the output.

The zero-knowledge proof (ZKP) [29] is a promising solution to prove the integrity of machine learning (ML) services. In ZKP, a prover produces a proof π to convince the verifier that the result of a public function f on public input x and the secret input of the prover w is indeed $y = f(x, w)$. The secret input w is usually referred to as the witness. ZKP guarantees that the verifier will output rejection with overwhelming probability if the prover cheats on calculation, while the proof π does not reveal any additional information about the secret w . Using f as the function that represents the calculation of the model, x as user input, and w as model weights within the ZKP protocol, the service provider can prove the correctness of LLM inference. In real-world applications, regulatory authorities can frequently and anonymously query the LLM service and require the service provider to prove the validity of the generated answers via ZKP. If the provider fails to provide the valid proof, they may face penalties. This approach effectively safeguards consumer rights while imposing small overhead on existing infrastructure.

In recent years, extensive research has been conducted on ZKP for ML, covering a range of models from classic decision trees to modern neural networks (NN). However, most of these works either do not support LLM architectures [9, 18, 19, 35, 37, 40, 61, 68], or support LLMs [10, 31, 41, 55] but face significant limitations, such as inefficiency and high communication overhead. For example, concurrent work [10] requires over an hour to generate a single proof for the inference of GPT-2 [50]. Such lengthy proof generation times render these approaches impractical for real-world applications involving LLMs. The core challenge in developing practical ZKP schemes for LLMs lies in efficiently proving different layers involved in LLMs, such as matrix multiplication, attention [57], GeLU [32], and normalization [5]. This challenge arises for two main reasons: (1) the linear layers in LLMs are

*Equal contribution.

†Corresponding author.

large in scale and frequently invoked, making their proof generation time consuming. (2) ZKP primarily operates within arithmetic fields that only support addition and multiplication operations, making it difficult to efficiently prove non-linear layers.

In this paper, we aim to address the challenge of designing efficient ZKPs for both linear and non-linear layers, with the goal of generating practical ZKPs for GPT models.

Efficient proof for linear layers. The linear layers in LLMs encounter weight matrices that are extremely large. This makes proving matrix multiplications time consuming. Therefore, we first focus on improving the efficiency of these proofs. Thaler et al. [56] proposed a dedicated protocol for verifiable matrix multiplication, which we incorporate into our system. While the algorithm achieves optimal prover time in theory, there remains significant room for improving its practical implementation. In particular, the execution time of this protocol is bottlenecked by the computation of the bookkeeping table for each matrix. The bookkeeping table stores evaluations of the multi-linear extension of each matrix column at a given random point. The classical method [40, 56] for this task utilizes the memory method in [58] to evaluate each column’s multi-linear extension. This requires approximately $4n \cdot m$ field multiplications per matrix, where n and m represent the number of rows and columns of the matrix, respectively. The major drawback of this method is that it does not leverage certain special properties of matrix elements to reduce computation as we have observed.

We propose a grouping algorithm to address this bottleneck efficiently, reducing the number of field multiplications involved in [56] to approximately half. We further accelerate our grouping algorithm by leveraging two key properties of our matrix elements: (1) the presence of many padded zeros due to sumcheck requirements, and (2) the relatively small range of values resulting from quantization. By utilizing our proposed method, the prover’s bottleneck shifts to $n \cdot m$ field additions. Since field multiplications are several times more expensive than field additions, our algorithm achieves nearly a tenfold acceleration on this theoretically optimal protocol.

Efficient proof for non-linear layers. Proving non-linear layers also poses challenges in ZKP due to the involvement of ZKP-unfriendly operations such as division, square root, and exponentiation. Previous work based on floating-point circuits [61] or numerical iterations [31] incurs high overhead when proving these operations. For example, [61] requires thousands of relations to prove division and exponentiation operations. The inefficiency of existing methods stems from simulating non-arithmetic computations using arithmetic operations. To address this issue, we propose to use the computation result as advice, instead of simulating the computation in the arithmetic field. We found that, with the use of advice, division and square root operations can be proved by a single range relation. For exponentiation operations, we prepare a lookup table containing all possible evaluations of the expo-

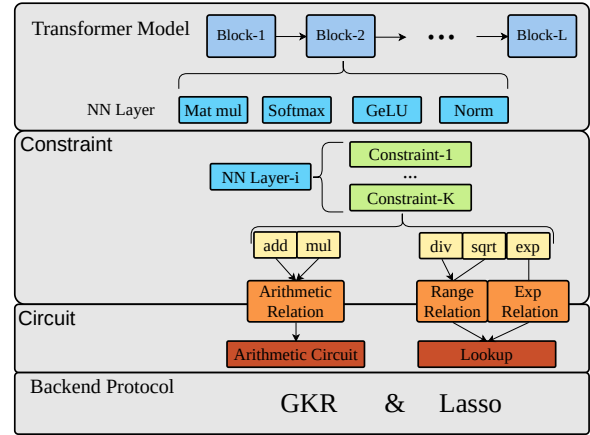


Figure 1: Workflow of the prover for LLM.

nentiation result. Then, each exponentiation operation can be verified with a single query to the lookup table. By leveraging advice, our approach reduces the number of relations required for proof by several orders of magnitude, significantly lowering the prover’s overhead for non-linear layers. To prove range relations and exponentiation operations, we employ the recent lookup scheme Lasso [53]. For proving the involved arithmetic relations, we adopt the GKR protocol [30]. By integrating the above proposed techniques, we have developed a proof system capable of efficiently proving different LLM layers.

Compared with existing ZKP schemes for LLMs [10, 31, 41, 55], our system offers significant advantages in the following aspects: (1) Prover efficiency. Our efficient design for proving both linear and non-linear layers greatly reduces the cost of proving LLMs. For example, [55] directly uses the protocol in [56] to prove matrix multiplication. Under the same settings, our system is nearly ten times faster than [55] in proving these layers due to the efficiency of our algorithm. Additionally, our system can prove all normalization layers in a GPT-2 model in less than 30 seconds, whereas Hao et al. [31] requires more than 4000 seconds. (2) Non-interactive and light communication. Our system can be easily transformed into a non-interactive proof using the Fiat-Shamir Transformation [20], enabling the proof to be downloaded and publicly verified offline. In contrast, previous VOLE-based solutions [31, 41] cannot be converted to non-interactive proofs. Additionally, VOLE-based solutions require gigabytes of data transfer between the prover and verifier, whereas our method is communication-efficient, with a proof size of 101 KB.

Performance improvement techniques. Although our system demonstrates several advantages, further optimizations are essential to ensure its applicability in real-world scenarios. Specifically, we aim to generate proof for GPT-2 inference within one minute on a CPU server. To achieve this, we perform comprehensive optimizations at both the constraint and circuit levels.

Constraints in ZKPs are essentially equations that define the computational relationships between variables. In our system, the constraints correspond to the computation process of the LLM. These constraints are then converted into arithmetic and non-arithmetic relations. Non-arithmetic relations include range and exponentiation relations. Finally, our backend proves these relations. The workflow is illustrated in Figure 1. Our improvements consist of two key components: constraint fusion and circuit squeeze.

At the constraint level, we propose a generalized optimization technique called **constraint fusion**. This technique aims to combine the computation of two adjacent rounding constraints, thereby reducing the overall number of rounding constraints. In our system, range relations are introduced for checking the correctness of rounding constraints; however, the large number of range relations becomes a bottleneck during the proof generation. By reducing the number of rounding constraints, we can alleviate the associated range relations, thereby mitigating prover overhead. Importantly, our constraint fusion technique preserves the original computational precision. After applying constraint fusion, we significantly reduce the proving cost of different LLM layers.

At the circuit level, we propose a technique called **circuit squeeze** to accelerate the prover. We observed existing GKR-based systems [6, 40] cannot effectively utilize multi-thread parallelism. Profiling reveals that the low utilization stems from the cost of thread synchronization during the sumcheck at each layer. Our circuit squeeze significantly enhances parallelism by reducing circuit depth and increasing the number of gates per layer. This optimization is possible due to the topological independence of subcircuits responsible for verifying each rounding constraint. This independence arises from our design choice to store the outputs of rounding computations as advice in the input layer, rather than computing them directly. For matrix multiplication layers not connected to standard GKR-style gates, we extend the protocol from [56] to support proving multiple matrix multiplications simultaneously. Our method provides substantial acceleration for the LLM circuit by eliminating sequential dependencies in proving computations across different NN layers.

Our key contributions are summarized as follows.

1. We are the first to present a practical ZKP design for LLMs under general settings. Our system overcomes the inefficiencies and limitations of existing works, particularly those related to large communication overhead and hardware requirements. The framework is compatible with LLMs utilizing prevalent architectures such as GPT-2 [50] and is capable of proving its inference under 25 seconds.
2. We propose efficient proof constructions for linear and non-linear layers. For linear layers, we propose a new algorithm to accelerate the matrix multiplication protocol in [56], reducing its concrete computation cost by orders of magnitude. For non-linear layers, we propose efficient methods based on advice to prove different ZKP-unfriendly operations. This reduces the proving cost by orders compared with existing prevalent solutions. To further enhance our system's efficiency, we propose constraint fusion, reducing the number of range relations to prove. Additionally, We propose circuit squeeze, which offers substantial parallelism acceleration by eliminating the sequential dependency in proving different NN layers.
3. We conduct extensive experiments on real-world CPU servers to validate the efficiency of our system. Results show that our implementation achieves over $279\times$ speedup compared to Hao et al. (USENIX Security'24) and $185\times$ speedup over the state-of-the-art Plonk-based approach [10] (Eurosys'24). Even compared to the concurrent VOLE-based scheme [41], our non-interactive system delivers a $5.1\times$ speedup in prover time and reduces the proof size by $23,000\times$.

2 Background

2.1 LLM and quantization

An LLM primarily consists of L stacked transformer blocks [57]. Each transformer block comprises various layers, including matrix multiplication, normalization (LLMs use layer normalization [5]), attention, and GeLU. The input to an LLM is an embedding matrix x of dimensions $s \times d$, where s is the length of text tokens, d denotes the hidden dimension size of the LLM. After x passes through the initial LLM transformer block, the output is a matrix f with dimensions $s \times d$. This matrix f is then passed to the next transformer block. The process repeats through all L blocks.

Real number calculations in ML models pose significant challenges for ZKP. Most ZKP approaches for ML [10, 18, 19, 35, 40, 41, 55] utilize quantization to convert real numbers to integers, making computations easier to prove. We adopt the quantization approach from zkCNN [40], where a real number x is mapped to an integer q using the formula $x = S(q - z)$. S denotes a floating-point number representing the quantization scale, and z is an integer known as the zero-point. All values of LLM parameters and intermediate results in computation are expressed as Q -bit integers q . The scale S and zero-point z are typically shared within a NN layer or tensor.

When two numbers are added and their scales are the same, we add their integer parts: $x_1 + x_2 = S(q_{x_1} - z_1 + q_{x_2} - z_2)$. However, when multiplying two numbers, we must perform **rescaling** to avoid overflow. Specifically, when multiplying $x = S_x q_x$ and $y = S_y q_y$ to obtain $z = S_z q_z$ (in this and subsequent derivations, we neglect the zero-point for simplicity), we compute $q_z = \text{round}\left(\frac{C_1}{C_2} \cdot q_x \cdot q_y\right)$, where C_1 and C_2 are Q -bit integers that ensure $\frac{C_1}{C_2} \approx \frac{S_x \cdot S_y}{S_z}$. This approach allows

floating-point addition and multiplication to be approximated through integer computation.

2.2 GKR for ML-friendly circuit

In our system, the computation steps of the quantized LLM are converted into constraints. And relations to prove are derived from constraints. These relations generally fall into two types. The first type is arithmetic relations, composed of additions and multiplications. We prove the arithmetic relations by converting them into an arithmetic circuit and using the GKR protocol [30]. The second type is non-arithmetic relations, such as range relations and exponentiation relations. They are proved by the Lasso protocol [53] introduced in Section 2.3.

The GKR protocol [30] uses the sumcheck protocol [42] as a building block to prove circuits consisting of addition and multiplication gates. Each gate takes at most two inputs from the previous layer and computes the result.

GKR protocol is widely adopted in ZKP due to its efficiency. However, the classical GKR protocol has a significant limitation under many ML scenarios: The weight matrices of ML models are committed in the input layer. Since each gate only receives input from the preceding layer, these weights must be relayed across the circuit to reach the desired layer. This process introduces numerous additional gates, leading to high computational overhead.

Define identity function $eq(x, y)$, which takes two bit-strings x and y of the same length as input. $eq(\cdot, \cdot)$ returns 1 when $x = y$; otherwise returns 0. Suppose $\tilde{eq}(\cdot, \cdot)$ is the multi-linear extension of $eq(\cdot, \cdot)$. $\tilde{eq}(x, y) = \prod_{i=1}^{\ell} ((1 - x_i)(1 - y_i) + x_i y_i)$.

Definition 1 (Multi-linear Extension [13]). Let $V : \{0, 1\}^{\ell} \rightarrow \mathbb{F}$ be a function. The multi-linear extension of V is the unique polynomial $\tilde{V} : \mathbb{F}^{\ell} \rightarrow \mathbb{F}$ such that $\tilde{V}(x_1, x_2, \dots, x_{\ell}) = V(x_1, x_2, \dots, x_{\ell})$ for all $x_1, x_2, \dots, x_{\ell} \in \{0, 1\}$. $\tilde{V}$ can be expressed as $\tilde{V}(x_1, x_2, \dots, x_{\ell}) = \sum_{b \in \{0, 1\}^{\ell}} \tilde{eq}(x, b) \cdot V(b)$.

To resolve the above limitation of GKR, [40, 69] introduced an elegant circuit construction alongside an extension of the GKR protocol. They modified the definitions of addition and multiplication gates to enable them to receive input either from the preceding layer or from the input layer directly. This reduces the need to relay values from the input layer. For more details, please refer to [40]. This particular circuit configuration is referred to as a "machine learning-friendly circuit" (ML-friendly circuit).

Following the notations in [40], we define a function $V_i : \{0, 1\}^{s_i} \rightarrow \mathbb{F}$ that takes a binary string b and returns the output of gate b in layer i . With this definition, V_0 corresponds to the input layer, and V_d corresponds to the circuit output. We denote the multi-linear extension of V_i as $\tilde{V}_i$. We also denote the subset of values in the input layer that connect to the i -th layer as $V_{i, in}$, with its multi-linear extension denoted as $\tilde{V}_{i, in}(\cdot)$.

At the i -th circuit layer, the prover and verifier invoke the sumcheck protocol to reduce two evaluations of $\tilde{V}_i$ to two evaluations of $\tilde{V}_{i-1}(\cdot)$ and two evaluations of $\tilde{V}_{i, in}(\cdot)$.

When reaching the input layer, the verifier has received two evaluations of $\tilde{V}_{i, in}(\cdot)$ for the input at each layer. To combine these evaluations into a single evaluation of the multi-linear extension of the input $\tilde{V}_{in}(\cdot)$, the prover and verifier invoke the sumcheck protocol.

2.3 Lookup protocol and range proof support

Lookup protocols are key components in ZKPs to prove non-arithmetic relations. They allow the prover to convince the verifier that a secret vector is contained within a public table. Specifically, for a vector $H \in \mathbb{F}^m$, the prover proves that each element of H exists within $T \in \mathbb{F}^n$, where T is a public table of size n , and m is the number of queries to the table.

In our approach, we utilize lookup protocols to prove exponentiation relations and range relations. When verifying the range relation $\ell_i \leq x_i \leq r_i$ for $i = 0, 1, \dots, m-1$, we can transform it into proving $0 \leq x_i - \ell_i \leq t$ and $0 \leq r_i - x_i \leq t$. ℓ_i and r_i are integers, and t is defined as the maximum value of $r_i - \ell_i$. We then construct a lookup table T containing numbers $0, 1, \dots, t$. Using the lookup protocol, we check that all $x_i - \ell_i$ and $r_i - x_i$ are in the table T , thus proving the m range relations. In our scenario, t may be quite large, potentially exceeding 2^{32} , resulting in a huge table size. Such large tables are infeasible for most existing lookup table approaches [16, 23, 66, 67], as these approaches either require prover time linear in the table size n or need preprocessing time of $O(n \log n)$.

However, a recent advancement in lookup protocols, Lasso [53], addresses this problem. When the table is structured—meaning that the query to the table can be decomposed into sub-queries to much smaller subtables—the prover's cost in Lasso, for any integer parameter $c > 1$, is dominated by committing to $3 \cdot c \cdot m + c \cdot n^{1/c}$ field elements. The table we use in our scheme to check range relations satisfies the structured table requirement in Lasso. Moreover, the exponentiation table utilized in our scheme is relatively small. Therefore, despite the exponentiation table being unstructured, Lasso achieves an efficient complexity of $O(n + m)$. For technical details of Lasso, please refer to [53]. For technical details of integrating Lasso into GKR, please refer to Appendix A.

3 Technical Overview

An LLM consists of many linear and non-linear layers. To ensure applicability in real-world scenarios, a practical ZKP system for LLMs should be able to efficiently prove both types of layers. In this paper, we design efficient proofs for linear and non-linear layers in LLMs.

Efficient proof for linear layers. Our proof of linear layers is based on the well-known protocol in [56] to prove matrix multiplication. This protocol is adopted directly in many

existing ZKP for ML systems [6, 40, 55]. It proves the multiplication of matrix A with shape $n \times m$ and matrix B with shape $m \times k$ results in matrix C . This protocol achieves a theoretical complexity of $O(nm + mk)$, which is optimal. However, we identified significant opportunities to improve its practical implementation by notably reducing the field operations required by the protocol.

The time cost of [56] is dominated by computing the bookkeeping table for the input matrices A and B . We propose a grouping algorithm to accelerate the computation of the bookkeeping table. Grouping performs many dot products between input matrix elements and precomputed $\tilde{e}q$ evaluations, effectively halving field multiplications.

Further optimization opportunities arise from leveraging the properties of matrix elements in our scenario. Due to the requirement of sumcheck, the matrix dimensions are padded to powers of 2 with zero. During each dot product computation, we skip these zero elements, saving numerous field operations. Additionally, due to quantization, all matrix field elements have relatively small values. Thus we accelerate computation by precomputing field multiplication results.

As a result of these optimizations, our final implementation of bookkeeping table computation for matrix A is bottlenecked by $n \cdot m$ field additions. In contrast, the classical implementation in [40, 56] consumes $2^{\lceil \log n \rceil + \lceil \log m \rceil + 1}$ field multiplications and $2^{\lceil \log n \rceil + \lceil \log m \rceil}$ field additions. In many cases, this amounts to approximately $4n \cdot m$ field multiplications. Given that field multiplications are several times more expensive than field additions [27, 44], our optimization is estimated to reduce the overhead of [56]’s protocol by nearly an order of magnitude.

Efficient proof for non-linear layers. Non-linear functions in LLMs are composed of division, square root, and exponentiation operations, which are not ZKP-friendly. Previous work [31, 61] proved these operations with hundreds or even thousands of relations, thus incurring high overhead. To resolve this inefficiency, inspired by the idea of checking advice correctness in [26, 28], we propose to check division, square root, and exponentiation in one non-arithmetic relation.

Take division as an example. The division of quantized x, y is $z = \text{round}(\frac{C_1}{C_2} \cdot \frac{q_x}{q_y})$, where C_1, C_2 are integers such that $\frac{C_1}{C_2} \approx \frac{S_x}{S_y \cdot S_z}$. We derive $C_2 q_y (2q_z - 1) \leq 2C_1 \cdot q_x < C_2 q_y (2q_z + 1)$. Similarly, square root can be transformed into **one** range relation. For exponentiation operations, we prepare a lookup table containing all possible evaluations of the exponentiation result. Then, each exponentiation operation can be verified through a single query to the lookup table. The range relations and lookup queries above are proved with the state-of-the-art Lasso [53] lookup protocol. For arithmetic relations, we compose them into an arithmetic circuit and prove them with the ML-friendly GKR protocol introduced in Section 2.2.

Now we are ready to prove each LLM layer using the previously designed building blocks. To further maximize the

prover efficiency, we introduce optimization methods at both the constraint and circuit levels.

Constraint fusion. Our first optimization idea is to merge adjacent rounding constraints to reduce the number of range relations to be proved. We discovered that proving range relations is the bottleneck in our system, as it is more costly than proving arithmetic relations. To address this issue, we combine computations across adjacent rounding constraints before rounding. Under appropriate conditions, merging these constraints reduces proving overhead by saving range relations to prove introduced by rounding. We developed rules to automatically determine the optimal merging strategies for different LLM layers.

For example, during the computation of the normalization layer, given an input vector x with length n , its mean $\mu = \frac{\sum_{i=1}^n x_i}{n}$ and variance $\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \mu)^2}{n}}$. Assume we want to prove σ is correct. [31, 41] proves the following constraints: (1) $q_\mu = \text{round}(\frac{\sum_{i=1}^n q_{x_i}}{n})$ (2) $q_t = \text{round}(\frac{\sum_{i=1}^n (q_{x_i} - q_\mu)^2}{n})$ (3) $q_\sigma = \text{round}(\sqrt{q_t})$. Based on these constraints, their systems can prove q_σ in **three** range relations.

However, leveraging the idea of merging adjacent rounding constraints, we can reduce the range relations that need to be proven to **one**. Specifically, we merge the constraints on q_μ and q_t , and substitute them into the last constraint on q_σ . We first turn q_μ and q_t into the analytic form: $q_\mu = \frac{\sum_{i=1}^n q_{x_i}}{n}$ and $q_t = \frac{\sum_{i=1}^n (q_{x_i} - q_\mu)^2}{n}$. By substituting them into q_σ , we have $q_\sigma = \text{round}(\sqrt{\frac{\sum_{i=1}^n (q_{x_i} - \frac{\sum_{i=1}^n q_{x_i}}{n})^2}{n}})$. We eventually derive the following range relation: $n(2q_\sigma - 1)^2 \leq \sum_{i=1}^n (nq_{x_i} - \sum_{i=1}^n q_{x_i})^2 < n(2q_\sigma + 1)^2$.

Circuit squeeze. We observe that our circuit, like existing GKR-based ZKP systems for ML [6, 38, 40], cannot effectively utilize multi-thread parallelism. This inefficiency comes from two main factors: (1) the small size of each circuit layer and (2) the Fiat-Shamir Transformation [20], which restricts parallelization to be conducted within each layer. As a result, significant time is wasted on synchronization due to the small sizes handled by each sumcheck.

Our optimization design stems from the fact that the computation order of the GKR protocol follows the circuit architecture. Thus, we have to reorganize the circuit architecture to improve parallelism opportunity. Our key observation is that due to our advice design, the subcircuits checking different constraints have no topological dependence. We can squeeze different subcircuits into the same set of layers leveraging this topological independence.

The remaining challenge is how to squeeze multiple matrix multiplication layers. This challenge arises because matrix multiplications are proven through the protocol in [56] rather than GKR-style sumcheck. To address this, we design a new protocol that simultaneously checks the correctness of multiple matrix multiplications based on sumcheck. Circuit

squeezing substantially enhances the performance of the LLM circuit by boosting the utilization of multi-thread parallelism.

4 Efficient Proof for LLM

4.1 Constraints for LLM layers

Utilizing the quantization scheme introduced in Section 2.1, we derive the constraints for different LLM layers.

Matrix multiplication. For matrix a and weight W , the matrix multiplication layer is computed as $b = a \cdot W$. The result matrix b needs to be rescaled to avoid overflow. The constraint is $q_b = \text{round}(\frac{S_a \cdot S_W}{S_b} a \cdot W)$.

Attention. The attention layer takes h matrix triples (Q_i, K_i, V_i) from each head as input and computes: $P = \text{Concat}(\text{Softmax}(\frac{Q_i \cdot K_i}{\sqrt{d}}) V_i)_{i=1, \dots, h}$. The computation of each head is data-independent before concatenation; for simplicity, we only write the constraints for a single head, and neglect the subscript for each head. Our constraints are listed as follows:

$$\begin{aligned} q_C &= \text{round}(\frac{S_Q \cdot S_K}{S_C} q_Q \cdot q_K) \\ q_E &= \text{round}(\frac{S_C}{S_E \sqrt{d}} \cdot q_C) \\ q_{E_{\max, i}} &= \max q_{E_{ij}}, \text{ for } i = 1, \dots, s \\ q_{e_{ij}} &= \exp(q_{E_{\max, i}} - q_{E_{ij}}), \text{ for } i = 1, \dots, s, j = 1, \dots, i \\ q_{s_{ij}} &= \text{round}(\frac{q_{e_{ij}}}{S_s \sum_{j=1}^i q_{e_{ij}}}), \text{ for } i = 1, \dots, s, j = 1, \dots, i \\ q_P &= \text{round}(\frac{S_s \cdot S_V}{S_P} q_s \cdot q_V) \end{aligned} \quad (1)$$

where $S_Q, S_K, S_V, S_C, S_E, S_s, S_P$ are the scales of $q_Q, q_K, q_V, q_C, q_E, q_s$ and q_P , respectively.

GeLU. GeLU is computed as $G = g \cdot \Phi(g)$. Directly proving it in ZKP is very challenging. Multiple approximations have been proposed but are not enough ZKP-friendly. We design a better approximation as $G = \frac{g+|g|}{2} - \mathbb{1}_{|g| > \text{th}} \cdot g \cdot (a \cdot |g|^2 + b \cdot |g| + c)$. For design details and comparisons, please see Appendix C. The constraints corresponding to our approximation are listed as follows:

$$\begin{aligned} s &= \mathbb{1}_{|q_g| \geq q_{\text{th}}} \\ q_P &= \text{round}(\frac{aS_g^2 q_g^2 + bS_g |q_g| + c}{S_P}) \\ q_D &= \text{round}(S_P \cdot q_g \cdot q_P) \\ q_G &= \frac{q_g + Q_g}{2} - s \cdot q_D \end{aligned} \quad (2)$$

where $Q_g = |q_g|$, q_{th}, q_G share scale S_g with q_g , $q_{\text{th}} = \text{round}(\text{th}/S_g)$.

Normalization. For matrix x , the normalization layer is computed as $y_{ij} = \gamma_j \cdot \frac{x_{ij} - \mu_i}{\sigma_i} + \beta_j$, where μ_i, σ_i is the mean and variance of row i of x , γ, β are vectors parameterizing the affine transform. The constraints are listed as follows:

$$\begin{aligned} q_{\mu_i} &= \text{round}(\sum_{j=1}^d q_{x_{ij}}/d) \\ q_{t_i} &= \text{round}(\sum_{j=1}^d (q_{x_{ij}} - q_{\mu_i})^2/d) \\ q_{\sigma_i} &= \text{round}(\sqrt{q_{t_i}}) \\ q_{s_i} &= \text{round}(\frac{q_{x_{ij}} - q_{\mu_i}}{S_s \cdot q_{\sigma_i}}) \\ q_{y_{ij}} &= \text{round}(\frac{S_s S_\gamma}{S_y} \cdot q_{\gamma_j} \cdot q_{s_i} + \frac{S_\gamma}{S_y} \cdot q_{\beta_j}) \end{aligned} \quad (3)$$

where μ, σ share scale S_x with q_x , β share scale S_γ with γ .

Due to the similarity in computational workflows, the constraints proposed in the concurrent work by Lu et al. [41] share some similarities with ours. The key distinction lies in our novel zk-friendly approximation for GeLU, referred to as z-GeLU, detailed in Appendix C. In contrast, Lu et al. [41] adopt the approximation from I-BERT [36] directly.

4.2 Efficient proof for linear layers

After deriving the LLM constraints, we now consider how to efficiently prove them. We first consider linear layers since they account for the majority of computation in plain-text LLM inference. We integrate the dedicated protocol for matrix multiplication in [56] into the ML-friendly GKR framework in a manner identical to that of [40]. Although the protocol achieves optimal prover time in theory, we identify significant opportunities to improve its concrete efficiency.

4.2.1 Analysis of classical implementation of [56]

The protocol of [56] is as follows: Denote the two input matrices as A, B , and the output matrix as C . Assume A, B, C have original shapes $n \times m, m \times k, n \times k$, respectively. Due to requirements of sumcheck, these matrices are all padded to powers of 2 in both width and height. [56]'s protocol uses the sumcheck protocol to reduce the claim on $\tilde{C}(z)$ to claims on $\tilde{A}(\cdot), \tilde{B}(\cdot)$. z is a random point chosen by the verifier. Denote $a||b$ as the concatenation of vectors a and b . Let $z = r||r'$, where $r \in \mathbb{F}^{[\log n]}, r' \in \mathbb{F}^{[\log k]}$. By the property of matrix multiplication, we have $\tilde{C}(r||r') = \sum_{c \in \{0,1\}^{[\log m]}} \tilde{A}(r||c) \tilde{B}(c||r')$.

Before running the sumcheck protocol on this equation, the prover first computes the bookkeeping tables: $\tilde{A}(r||c)$ and $\tilde{B}(c||r')$ for all $c \in \{0,1\}^{[\log m]}$. Existing

Precompute		Precompute		Matrix Element
$\tilde{e}q(x, 00)$	$\times$	$\tilde{e}q(y, 00)$	$\times$	$\tilde{A}(0000 c)$
		$\tilde{e}q(y, 01)$	$\times$	$\tilde{A}(0001 c)$
		$\tilde{e}q(y, 10)$	$\times$	$\tilde{A}(0010 c)$
		$\tilde{e}q(y, 11)$	$\times$	$\tilde{A}(0011 c)$
$\tilde{e}q(x, 01)$	$\times$	$\tilde{e}q(y, 00)$	$\times$	$\tilde{A}(0100 c)$
		$\tilde{e}q(y, 01)$	$\times$	$\tilde{A}(0101 c)$
		$\tilde{e}q(y, 10)$	$\times$	$\tilde{A}(0110 c)$
		$\tilde{e}q(y, 11)$	$\times$	$\tilde{A}(0111 c)$
...	...	...	...	...

Figure 2: Intuition of grouping algorithm for computing the bookkeeping table.

methods [40, 56] leverage the following equation to iteratively compute $\tilde{A}(r||c)$: $\tilde{A}(r_1, \dots, r_i, b_{i+1}, \dots, b_{\lceil \log n \rceil} || c) = (1 - r_i) \tilde{A}(r_1, \dots, r_{i-1}, 0, b_{i+1}, \dots, b_{\lceil \log n \rceil} || c) + r_i \tilde{A}(r_1, \dots, r_{i-1}, 1, b_{i+1}, \dots, b_{\lceil \log n \rceil} || c)$. This is identical to the "memorization" method in [58] to evaluate multi-linear extension. The method's computation cost for each c is $2 \cdot (2^{\lceil \log n \rceil - 1} + \dots + 1) \approx 2^{\lceil \log n \rceil + 1}$ field multiplications and $2^{\lceil \log n \rceil}$ field additions. Therefore, the total cost for computing $\tilde{A}(r||\cdot)$ is $2^{\lceil \log n \rceil + \lceil \log m \rceil + 1}$ field multiplications and $2^{\lceil \log n \rceil + \lceil \log m \rceil}$ field additions.

4.2.2 Optimizations based on grouping and matrix element properties

Next, we will introduce our efficient algorithm for computing the bookkeeping table.

Efficient computation by grouping. By the definition of multi-linear extension, we have $\tilde{A}(r||c) = \sum_{b \in \{0,1\}^{\lceil \log n \rceil}} \tilde{e}q(r, b) \tilde{A}(b||c)$. Without loss of generality, assume $\lceil \log n \rceil$ is even. Let $r = x||y, b = L||R$, where $x, y, L, R \in \mathbb{F}^{\frac{\lceil \log n \rceil}{2}}$. We observe that $\tilde{e}q(\cdot)$: $\tilde{e}q(r, b) = \tilde{e}q(x, L) \cdot \tilde{e}q(y, R)$. Leveraging this property, we have:

$$\tilde{A}(r||c) = \sum_{L \in \{0,1\}^{\frac{\lceil \log n \rceil}{2}}} \tilde{e}q(x, L) \cdot \left(\sum_{R \in \{0,1\}^{\frac{\lceil \log n \rceil}{2}}} \tilde{e}q(y, R) \cdot \tilde{A}(L||R||c) \right) \quad (4)$$

Computing using the above equation is more efficient. The procedure is as follows: All $\tilde{e}q(x, \cdot)$ and $\tilde{e}q(y, \cdot)$ are precomputed once in $O(2^{\frac{\lceil \log n \rceil}{2}})$ field operations. Imagine there are $2^{\frac{\lceil \log n \rceil}{2}}$ groups represented by different L . Computing Equation 4 could be viewed as conducting a dot product between $\tilde{e}q(y, \cdot)$ and $\tilde{A}(L, \cdot || c)$ in each group; then the result of the dot product in each group forms a vector; at last this vector performs a dot product with $\tilde{e}q(x, \cdot)$ to obtain $\tilde{A}(r||c)$. An intuitive example can be found in Figure 2.

Skiping computation by leveraging sparsity. The benefit of leveraging grouping is that we can efficiently exploit the

properties of matrix A , since the elements of A are directly used in each inner dot product. Specifically, during the computation of each inner dot product, we can skip over elements in A that are zeros, thereby reducing the number of field operations required by our method. This opportunity arises because matrix A is padded with many zero elements to satisfy the requirement of sumcheck.

Saving field multiplication by precomputing. We also observe another important property of matrix A is that elements in A are all quantized numbers. This means that all of their value satisfy $\in [0, 2^Q]$, where $2^Q \ll |\mathbb{F}|$. Thus, the multiplication between each $\tilde{e}q(y, \cdot)$ and different $\tilde{A}(b||c)$ only obtains at most $2^Q + 1$ different results. Furthermore, these possible results are $\{0, 1, \dots, 2^Q\}$ times of $\tilde{e}q(y, b_{\frac{\lceil \log n \rceil}{2}+1} \dots b_{\lceil \log n \rceil})$. We can efficiently derive all these results by only performing simple field additions on $\tilde{e}q(y, b_{\frac{\lceil \log n \rceil}{2}+1} \dots b_{\lceil \log n \rceil})$.

Cost analysis. The full algorithm for computing the bookkeeping table for matrix A is shown in Algorithm 1. The two bottlenecks in our computation is precomputing the field multiplication results in line 10 and performing the inner dot product based on precomputed results in line 18. These two parts each correspond to $2^{\lceil \frac{\log n \rceil}{2} + Q}$ field additions and $n \cdot m$ field additions. Our optimizations can significantly mitigate computation overhead compared with existing implementation [56]. Take computing the bookkeeping table of the matrix of shape 3072×768 in the GPT-2 feedforward layer as an example. Based on previous analysis, the classical method costs 8.4×10^6 field multiplications and 4.2×10^6 field additions. Substituting $Q = 16$, Our optimized method costs $n \cdot m + 2^{\lceil \frac{\log n \rceil}{2} + Q} \approx 4.5 \times 10^6$ field additions. Assume one field multiplication is 5 times cost of one field addition. Our method is $10.2 \times$ more efficient than the classical method. The optimization techniques presented in our work could also be used to accelerate the evaluation of multi-linear extensions under other scenarios in ZKP.

4.3 Efficient proof for non-linear layers

Proving non-linear layers in the LLM also poses a significant challenge. The challenge comes from these layers invoking ZKP-unfriendly operations such as division, square root, and exponentiation. Existing approaches either simulate these operations in the circuit—using binary decomposition and floating-point emulation [40, 61] or proving numerical iterations [17, 31]. These approaches all incur a large number of relations to prove and significant overhead. For example, [61] requires thousands of relations to prove division and exponentiation operations.

Recent advances in ZKP systems [4, 26] leverage advice to efficiently prove non-arithmetic operations. [4] proves ZKP-unfriendly bitwise operations like XOR by taking the result as advice and querying it in the lookup table. This is much cheaper than simulating XOR in a binary circuit. Inspired

by these works, we propose using the computation result of division, square root, and exponentiation as advice and check the correctness of the computation. This is much cheaper than simulating the computation in arithmetic relations.

For the division operation $z = \frac{x}{y}$, the actual constraint is $q_z = \text{round}(\frac{C_1}{C_2} \cdot \frac{q_x}{q_y})$. We approximate $\frac{S_x}{S_y \cdot S_z}$ using $\frac{C_1}{C_2}$, C_1 and C_2 are integers. C_1, C_2 are obtained by searching in an interval $[1, 2^Q]$ and finding the nearest fraction to $\frac{S_x}{S_y \cdot S_z}$. By the property of rounding, we have $q_z - \frac{1}{2} \leq \frac{C_1}{C_2} \cdot \frac{q_x}{q_y} < q_z + \frac{1}{2}$. Thus, $C_2 q_y (2q_z - 1) \leq 2C_1 \cdot q_x \leq C_2 q_y (2q_z + 1) - 1$. Similarly, we can convert proving square root operation $q_y = \text{round}(\sqrt{q_x})$ into a range relation: $(2q_y - 1)^2 \leq 4q_x \leq (2q_y + 1)^2 - 1$. Therefore, for division and square root operations, we can transform checking their correctness into a single range relation. For proving exponentiation operations, we construct a lookup table containing pairs of input and output $(q_t, \text{round}(\frac{e^{S_t \cdot q_t}}{S_z}))$. Exponentiation operation $q_z = \text{round}(\frac{e^{S_t \cdot q_t}}{S_z})$ can be proved by querying (q_t, q_z) in this table, checking whether the pair belongs to the table. By leveraging the Lasso lookup protocol, we can efficiently prove these non-arithmetic relations. Experiment results show that our designs save the time cost of proving non-linear layers by several orders.

Proving non-linear layers in the LLM also involves proving some arithmetic relations. We assemble these relations into an arithmetic circuit and then prove the circuit using the ML-friendly GKR protocol.

4.4 Putting everything together

With the building blocks presented in the previous sections, we construct a ZKP protocol for LLM. The values in the LLM circuit's input layer consist of public input and secret witnesses. Following previous works [41, 55], our LLM circuit takes the input embedding matrix as public input. The witness includes model weight, intermediate results obtained by rounding, and queries to every lookup table. The prover commits to all witness values. The commitment to weight is done once in preprocessing and reused across all users. The intermediate results and lookup query values depend on user input. Thus, their commitment are computed for each user separately. The verifier runs the Lasso protocol with the prover to check whether the non-arithmetic relations are satisfied. Then, the prover and verifier run the GKR protocol on the arithmetic circuit, checking all the arithmetic relations between the variables. For the specified matrix multiplication layer, it is proven by the dedicated protocol in [56] leveraging our optimized algorithm in Section 4.2. After the GKR protocol, the prover and verifier reduce the claim to the correctness of a claim at the input layer. They open the commitment to the input layer values to determine whether the protocol passes. The compliance of all the relations ensures the correctness of

the whole LLM computation. The entire protocol is detailed in Protocol 1 in Appendix A. The proofs of soundness and completeness for the zkGPT protocol are provided in Appendix B. The zero-knowledge property can be incorporated into the protocol by using masking polynomials in a manner similar to that described in [69].

5 Constraint level optimization

In the previous sections we have built a non-interactive ZKP system for LLM. To ensure better applicability in real-world scenarios, we illustrate how we optimize our system at the constraint level in this section.

Previous works most related to our research [35, 40] introduced a technique to fuse the rescaling division after convolution with the subsequent ReLU constraint, reducing the number of relations to prove. However, they did not analyze constraint fusion opportunities in general cases, limiting the method's applicability to other NNs.

5.1 Constraint level optimization opportunity

In Section 4.1, we derive the constraints for each LLM layer. Rounding is incorporated in these constraints to ensure that intermediate results remain within the integer domain. However, the mapping of LLM layers to constraint sets is not unique; different approximations can lead to multiple sets of constraints, each with varying proving costs. These costs are largely influenced by the number of roundings, as rounding introduces range relations to prove, which are more computationally expensive than proving arithmetic relations.

Constraint fusion draws inspiration from operator fusion [11, 46], a well-known method for accelerating NN executions. The core idea of constraint fusion is to consolidate various constraints into a more concise structure. In particular, the approach involves merging computations within two adjacent rounding operations, resulting in one merged rounding constraint, thereby decreasing the overhead of proving range relations.

However, assessing the benefits of merging constraints is challenging. Some merges can reduce the number of roundings at free cost. Some may even significantly increase overhead. Additionally, some merges might not immediately reduce roundings but could pave the way for more effective merges later. The diverse and varying forms of constraints across computations further complicate manual analysis.

5.2 Constraint merging rules

Constraint classification. To better analyze the benefit of merging constraints inside the constraint, we classify the existing constraints with rounding into several types:

- Type-I: constraints that only contain arithmetic computation (Add: $q_y = q_{x_1} + q_{x_2}$, Mul: $q_y = q_{x_1} * q_{x_2}$).

Table 1: Merging different types of constraints. Whether merging constraint i with constraint j is profitable (denoted as P), potentially beneficial (denoted as PB), or unprofitable (denoted as UP).

First const. \ Second const.	I	II	III	IV
I	PB	PB	PB	UP
II	PB	P	P	UP
III	PB	P	P	UP
IV	UP	UP	UP	UP

- Type-II: constraints that contain division mixed with arithmetic computation. A typical representation is $q_y = \text{round}(\frac{S_{x1}S_{x2}}{S_y}q_{x1}q_{x2})$ (we omit the addition for simplicity).
- Type-III: constraints that contain square root mixed with division and arithmetic computation. The simplified representation is $y = \text{round}(\sqrt{t})$, where t is Type-I or Type-II constraint.
- Type-IV: constraints that contain exponentiations.

Type-I and Type-II constraints are used in all LLM layers. Type-III constraint is used in Normalization. Type-IV is employed by softmax function in attention layer.

Table 1 presents all potential constraint merging combinations. A profitable merge results in performance gains by reducing the number of rounding constraints while introducing minimal or no overhead on the arithmetic circuit side. A potentially beneficial merging offers opportunities for improvement, though it may not directly reduce rounding or increase proving costs. An unprofitable merge harms performance as the introduced overhead outweighs the savings from reducing the number of rounding constraints. We perform constraint merging only for profitable or potentially beneficial cases, leaving unprofitable ones untouched. The constraint type may change with fusion, but it remains one of the four fundamental types. In the following, we elaborate on profit analysis of representative combinations in Table 1.

Merging Type-II with itself is profitable. Merging Type-II constraints reduces the number of range relations to prove, and does not introduce extra overhead on the arithmetic circuit side.

Merging Type-II with Type-III is profitable. For instance, assume the first constraint is $q_y = \text{round}(\frac{C_{x1x2}}{C_y}q_{x1}q_{x2})$, where C_{x1x2} and C_y are integers. And the second constraint is $q_z = \text{round}(\sqrt{q_y})$. After merging, the rounding constraint derives one range relations forming $(2q_z - 1)^2C_y \leq 4C_{x1x2}q_{x1}q_{x2} \leq (2q_z + 1)^2C_y$. Although merging introduces small overhead due to the arithmetic circuit (square), it is minimal compared to the rounding saved.

Merging Type-III with Type-II or Type-III is profitable.

Consider the first constraint $q_y = \text{round}(\sqrt{q_x})$ and the second constraint $q_z = \text{round}(\frac{C_{yyt}}{C_zq_y}q_{yt})$, where C_{yyt} and C_z are integers and $\frac{C_{yyt}}{C_z} \approx \frac{S_yS_{yt}}{S_z}$. The rounding constraint derives to $(2q_z - 1)^2C_z^2q_x \leq 4C_{yyt}^2q_{yt}^2 \leq (2q_z + 1)^2C_z^2q_x$. Although extra square computation introduce arithmetic overhead, it is still minimal compared to the rounding saved. Similar with merging Type-III with Type-II, merging Type-III with Type-III is profitable too.

Merging Type-I with others (except Type-IV) or merging others (except Type-IV) with Type-I is potentially beneficial. It is easy to see that merging Type-I with other constraints does not introduce extra gate or change the rounding constraints since Type-I constraint only contain arithmetic computation. While merging does not directly eliminate rounding, it creates opportunities for further merging with adjacent constraints, potentially reducing overall complexity and overhead.

Merging with Type-IV or Merging Type-IV with others is unprofitable. For the first case, since the query for the lookup table must be an integer, rounding can not be eliminated for this case. For the second case, it prevents reusing the same exponentiation table across different LLM layers, leading to a high commitment cost for multiple extra tables and diluting the final performance.

Sequential merging strategy. Given a group of constraints, we sequentially merge adjacent constraints when the merging is either profitable (P) or potentially beneficial (PB). This merging process is repeated until no further profitable merges are possible. The final set of constraints remains consistent regardless of the merging order.

Error analysis. Assume we have constraint $q_y = \text{round}(F(q_x))$, $q_z = \text{round}(G(q_y))$, F, G are arbitrary real number functions. Denote the merged constraint as $q_{z'} = \text{round}(G(F(q_x)))$. Denote $p = G(F(q_x))$. We find that $|q_{z'} - p| \leq |q_z - p|$. This is because of, $q_{z'}$ equals $\text{round}(p)$. Thus it is the integer that minimizes $|q_{z'} - p|$. Since q_z is also belongs to the integer domain, $|q_{z'} - p| \leq |q_z - p|$ must hold. This means that constraint fusion never increases the numerical error of quantization. In contrast, it often reduces this error.

5.3 Optimized constraint after merging

After conducting sequential merging for each LLM layer, we can get appropriate constraint groups.

Attention. After merging q_C with q_E and q_{sij} with q_{Pij} , we are able to effectively reduce the number of rounding constraints of attention from $\frac{3}{2}hs(s+1) + sd$ to $\frac{1}{2}hs(s+1) + sd$. This reduction results in a nearly 30% decrease in the total number of rounding constraints required compared to the original attention.

$$\begin{aligned}
q_E &= \text{round}\left(\frac{S_Q \cdot S_K}{S_E \sqrt{d}} \cdot q_Q \cdot q_K\right) \\
q_{E_{\max,i}} &= \max q_{E_{ij}}, \text{ for } i = 1, \dots, s \\
q_{e_{ij}} &= \exp(q_{E_{\max,i}} - q_{E_{ij}}), \text{ for } i = 1, \dots, s, j = 1, \dots, i \\
q_{P_{ij}} &= \text{round}\left(\frac{S_V(q_e \cdot q_V)_{ij}}{S_P \sum_{k=1}^i q_{e_{ik}}}\right)
\end{aligned} \tag{5}$$

GELU After merging q_D with q_G , we are able to effectively reduce the number of GeLU rounding constraints from $2sd$ to sd . This reduction results in a nearly 50% decrease in the total number of rounding constraints required compared to the original GeLU.

$$\begin{aligned}
s &= \mathbb{1}_{|q_x| \geq q_{\text{th}}} \\
q_g &= \text{round}\left(\frac{q_x + Q_x}{2} - s \cdot (Q_x(a \cdot S_x^2 q_x^2 + b \cdot S_x Q_x + c))\right)
\end{aligned} \tag{6}$$

Normalization. The merging process for Normalization differs slightly from the usual strategy; After merging q_{μ_i} with q_{t_i} and q_{s_i} , we did not merge q_{σ_i} with $q_{y_{ij}}$ to prevent increasing the arithmetic and commitment overhead by a factor of N . As a result, we reduce the number of rounding constraints for normalization from $3s + 2sd$ to $s + sd$. This leads to nearly a 50% decrease in the total number of rounding constraints compared to the original normalization.

$$\begin{aligned}
q_{\sigma_i} &= \text{round}\left(\sqrt{\frac{\sum_{j=1}^d (q_{x_{ij}} \cdot d - \sum_{j=1}^d q_{x_{ij}})^2}{d^2}}\right) \\
q_{y_{ij}} &= \text{round}\left(\frac{S_\gamma}{S_y} \cdot q_{\gamma_j} \cdot \frac{q_{x_{ij}} \cdot d - \sum_{j=1}^d q_{x_{ij}}}{q_{\sigma_i} \cdot d} + \frac{S_\gamma}{S_y} \cdot q_{\beta_j}\right)
\end{aligned} \tag{7}$$

For a detailed explanation of the rounding savings effect from constraint fusion, readers are referred to Table 7 in the Appendix.

6 Circuit Optimization

A persistent challenge in ZKP is maximizing the utilization of computational resources during proof generation. ZKP parallelization [39, 62, 65] offers a promising solution. However, existing ZKP systems for ML [6, 38, 40, 41, 61] fail to fully exploit parallelism. We address this limitation by leveraging the insight that the LLM execution trace is already known during proving. Consequently, proving LLM execution does not need to follow the sequential order of LLM computation.

6.1 Circuit level optimization opportunity

We discover that existing ZKP for ML approaches [6, 31, 38, 40, 41] achieve poor speedup under multi-thread parallelism.

For sVOLE-based systems [31, 41], high communication overhead undermines the parallelism opportunity. As profiled in emp-zk [12], doubling computational resources yields less than 10% speedup under limited network bandwidth.

Although GKR-based systems [6, 38, 40] do not require network communication, they still suffer from low parallelism utilization. In the GKR protocol, parallelization is conducted within the sumcheck protocol of each layer. Different entries of the bookkeeping table are computed using multi-threaded parallelism by leveraging the "distributed sumcheck" paradigm proposed in [65]. The parallelization is limited within the sumcheck of each layer due to the Fiat-Shamir Transformation. Note that NNs typically consist of many layers, each with several constraints. Thus, for proving an NN circuit, the computation required for sumcheck in each layer is relatively small. Consequently, the synchronization process occupies a large proportion of computation time, leading to poor speedup.

Based on previous observations, our optimization is to squeeze the circuit into a wider and shallower shape. However, circuit squeezing faces the challenge that GKR circuits naturally impose layer-wise dependency. If variable b is computed based on variable a , then b has to be placed in subsequent layers of a . This makes it seem impossible to squeeze the circuit without making large changes to the circuit first.

6.2 Acceleration by circuit squeeze

The core idea of circuit squeeze is to break the layer-wise dependency based on commitment. Then, multiple subcircuits are topologically independent and could be squeezed into the same set of layers. Circuit squeezing helps reduce the circuit depth and increase the number of gates per layer, offering much better opportunities for parallelism.

Zero overhead circuit squeeze. Inspired by the construction of the Plonk [24] proof system, which has no sequential dependency in proving different constraints, we discover that committing intermediate results in the input layer can eliminate the topological dependency in the GKR circuit.

Since each subcircuit checks constraints using advice to reduce overhead, no additional commitment effort is required to squeeze these subcircuits. Moreover, as each subcircuit directly takes input from the input layer, they are topologically independent. Leveraging this independence, multiple subcircuits can be combined into a single set of layers, thereby restructuring the circuit into a wider and shallower form.

In general, we can apply circuit squeezing as follows: for each circuit layer that checks the correctness of a rounding constraint, starting from the i -th layer ($i > 1$), we move each gate originally in layer j to the $(j - i + 1)$ -th layer, while maintaining the connections between these gates. For our LLM circuit, since intermediate results are already committed as advice, squeezing incurs zero overhead.

Circuit squeeze for matrix multiplication. However, the

above mentioned technique to merge layers comprised of normal GKR gates is not applicable to matrix multiplication layers. Because these layers are proved using the dedicated protocol [56]. And the protocol in [56] only supports proving one matrix multiplication in each invocation.

To solve this problem, we design a new protocol generalizing [56] to support proving multiple matrix multiplication. Assume the circuit layer we consider is layer 1, and the number of matrix multiplications is M . After execution of GKR on previous layers, we have claims $\tilde{V}_1(r_1)$ and $\tilde{V}_1(r_2)$.

Inspired by the method in [6] proving multi-channel convolution in sumcheck, we can check multiple matrix multiplications by concatenating the matrices and checking the "batched" matrix multiplication relation using sumcheck. Denote the two matrices involving in the i -th matrix multiplication as A_i, B_i , the output of the i -th matrix multiplication as C_i . Assume all A_i are padded to the same shape, with each dimension to the power of 2. B_i is padded in the same manner. Denote A_i has shape $n \times m$, B_i has shape $m \times k$, C_i has shape $n \times k$. Assume the number of matrix multiplications M is also padded to the power of 2. Denote the concatenation of A_i as $A \in \mathbb{F}^{M \times n \times m}$, $B \in \mathbb{F}^{M \times m \times k}$ is defined similarly. $V_1 \in \mathbb{F}^{M \times n \times k}$ is the flattened form of the concatenation of C_i . For $\forall i \in \{0, 1\}^{\log M}, j \in \{0, 1\}^{\log n}, p \in \{0, 1\}^{\log k}$, we have $V_1(i||j||p) = \sum_{c \in \{0, 1\}^{\log m}} A(i||j||c) \cdot B(i||c||p)$.

The above equation is well-suited for proving within the sumcheck protocol. Let $r_1 = r' || r'_x || r'_y, r_2 = r'' || r''_x || r''_y$, where $r', r'' \in \mathbb{F}^{\log M}, r'_x, r''_x \in \mathbb{F}^{\log n}, r'_y, r''_y \in \mathbb{F}^{\log k}$. We have the following:

$$\begin{aligned} & \alpha \cdot \tilde{V}_1(r_1) + \beta \cdot \tilde{V}_1(r_2) \\ &= \sum_{c \in \{0, 1\}^{\log m}} \alpha \tilde{A}(r' || r'_x || c) \cdot \tilde{B}(r' || c || r'_y) + \beta \tilde{A}(r'' || r''_x || c) \cdot \tilde{B}(r'' || c || r''_y) \end{aligned} \quad (8)$$

Running sumcheck on the above equation, we can reduce claims on $\tilde{V}_1(r_1)$ and $\tilde{V}_1(r_2)$ to $\tilde{A}(\cdot)$ and $\tilde{B}(\cdot)$. The remaining challenge is whether we can generalize the efficient method to run the protocol in [56] in Section 4.2 to our new protocol. Essentially, we need to efficiently compute $\tilde{A}(r' || r'_x || c), \tilde{B}(r' || c || r'_y), \tilde{A}(r'' || r''_x || c), \tilde{B}(r'' || c || r''_y)$ for each $c \in \{0, 1\}^{\log m}$. By symmetry, we only need to consider how to efficiently compute $\tilde{A}(r' || r'_x || c)$. By the property of multi-linear extension, we have $\tilde{A}(r' || r'_x || c) = \sum_{b' \in \{0, 1\}^{\log M + \log n}} \tilde{e}q(r' || r'_x || b') \cdot \tilde{A}(b' || c)$.

The equation above is similar in form to the bookkeeping table used in the matrix multiplication protocol of [56]. Leveraging techniques similar to Algorithm 1 described in Section 4.2, we can compute the above equation efficiently. Due to space constraints, we omit the details.

Table 2: Acceleration of matrix optimizations.

	$(32 \times 768) \cdot (768 \times 3072)$	Optimized	GPT-2 matrices	Optimized
Time	0.25s	0.037s	9.8s	1.5s

7 Experiment

In this section, we present a comprehensive evaluation of our ZKP framework for LLM.

7.1 Implementations

Software. Our work is mainly implemented in C++. Our implementations of sum-check and ML-friendly GKR are based on the open source implementation of zkCNN [40]. We use Hyrax [59] as our polynomial commitment scheme because of its efficient prover time, reasonable proof size, and transparency. The prover time is $O(N)$ and the proof size and the verifier time are $O(\sqrt{N})$ for a polynomial of size N . We utilize the mcl library* to implement operations on finite field and BN254 elliptic curve adopted in [7, 39]. BN254 provides around 100 bits of security.

Hardware. Our evaluations are performed on a server equipped with an Intel Xeon 6126 2.60GHz 16-Core CPU and 200GB of memory. We leverage 32 thread parallelization by default.

Implementation acceleration. We accelerate the prover's commitment of the circuit's input values using multi-thread parallelism and Pippenger algorithm [49]. Each sumcheck involved in the GKR protocol is accelerated by multi-thread parallelism.

Quantization details. In our experiments, we use quantization level $Q = 16$. This means that all quantized values q belongs to range $[0, 65536]$. For ensuring value bound, the prover proves that the committed quantized weight lies within the range $[0, 2^Q]$, while the quantized input embedding is public and easy to verify. Rescaling with appropriate scaling factor is performed at each layer to ensure the computation results always fall the range $[0, 2^Q]$.

Baselines. In this paper, we compare the performance of our system with existing ZKP systems for LLM [10, 31, 41, 55]. [10] and [41] natively supports multi-thread parallelism thus we directly use their implementation for comparison. The proof system of [55] is purely based on GPU parallelism implemented by CUDA. Thus, we run it on a server with NVIDIA A100 GPU. For the VOLE-based schemes [31, 41], we evaluate performance under a standard network environment with 500 Mbps bandwidth.

7.2 Performance of zkGPT

7.2.1 Performance of linear layers

We first benchmark the performance of our efficient algorithm for accelerating proof of matrix multiplications under single thread. The results can be found in Table 2.

We can see that our algorithm does achieve very significant optimization effect compared with the original algorithm in [56]. For the case of proving matrix multiplication happening in the GPT-2 feed-forward network, our method achieves an acceleration ratio of $6.8\times$.

For a single transformer block in GPT-2, it contains 4 matrix multiplications, each weight matrix has shape $768 \times 2304, 768 \times 768, 768 \times 3024, 3024 \times 768$, respectively. We also profile the overhead of proving all of these matrix multiplications happening in GPT-2 inference in Table 2. We can see that, proving these multiplications utilizing the classical method in [56] requires about 10 seconds, which is quite considerable compared with our goal of squeezing our prover time of GPT-2 inference into one minute. Leveraging our optimizations, the time cost of proving all multiplications in GPT-2 can be reduced by $6.5\times$. This highlights the significance of our efficient method for proving matrix multiplications.

7.2.2 Overall performance

Next, we compare our system with existing ZKP systems for LLM. We also measure the LLM functionality loss caused by quantization. From Table 3, we can see that even under single thread setting and without our optimizations, our system achieves quite impressive performance compared to existing work. Compared with one of the state-of-the-art non-interactive proof schemes ZKML [10] which requires more than 1 hour to generate a proof for GPT-2, our system is $13\times$ more efficient, consuming approximately 5 minutes. The major reason is that our system is based on GKR and avoids the expensive FFT operations in Plonk-based systems. Comparing with state-of-the-art VOLE based scheme [41], though our system's prover time is a few times longer without multi-thread parallelism and optimizations, our scheme is non-interactive and the proof size is smaller by two orders of magnitude. This reduces the need of heavy communication burden for the verifier, also enabling the proof to be verified by the public.

After applying our proposed optimizations and multi-thread parallelism, the advantage of our system is amplified. Table 3 shows that our fully optimized system is $185\times$ more efficient than the state-of-the-art non-interactive scheme ZKML, and $5.1\times$ more efficient than the state-of-the-art VOLE-based scheme Lu et al [41], while our communication is $23000\times$ more efficient. Note that in MLaaS settings, the verifier normally owns much less computation power.

*<https://github.com/herumi/mcl>

Table 3: Time comparison of different proof systems for GPT-2 model.

Scheme	Prover time	Verifier Time	Proof size
Hao et al [31] [†]	6096s		6.85 GB
ZKML [10] (32 thread)	4026s	12.1s	7.8 K
Lu et al [41] [‡] (32 thread)	112.3s	31.4s	2.24 GB
zkLLM [55] (6912 CUDA cores)	15.8s	0.54s	126K
Ours (Single thread) wo all opt	319.6s	0.46s	154K
Ours (32 thread) wo all opt	52.3s	0.46s	154K
Ours (32 thread) w all opt	21.8s	0.35s	101K

Table 4: Generate text quality of GPT-2 and our quantization on various datasets.

Dataset	WikiText2	PTB	LAMBADA
FP32	29.3	41.3	48.4
Our quantization	29.5	41.7	48.7

When comparing with zkLLM [55], though our concrete prover performance is 30% slower, we mainly attribute it to different hardware configurations. CPUs generally have inferior parallel computing power compared with GPUs. For the 16 core CPU we use in our experiments, it has 10.2 TFLOPS computing power in total. While for A100 GPU, its computing power is 312 TFLOPS when fully utilized [47], which is $30\times$ stronger. This indicates that our system utilizes computation power in a much more effective way. This roots in multiple reasons (1) zkLLM proves each transformer block sequentially; Due to our circuit squeezing technique, our system has better parallelization opportunity than zkLLM. (2) zkLLM directly adopts the matrix multiplication protocol in [56], while our optimized method consumes much less field operations. (3) zkLLM did not propose techniques to reduce the number of non-arithmetic relations to prove from the constraint level. Thus our system is very likely to outperform them under the same hardware.

We also assess the functional degradation resulting from quantization. Following previous works in quantization [15, 64], we leverage the perplexity (PPL) [34] metric computed on multiple datasets. This metric measures the quality of LLM generated text, lower PPL indicated higher quality. We employ WikiText-2 [45], PTB corpus [43], and LAMBADA dataset [48], which are all commonly used benchmarks in LLM quantization [14, 36, 64]. On the three datasets, we can see that, the increase on PPL are all less than 0.5. This indicates that the LLM functionality loss introduced by our quantization is very small.

[†] [31] is not open-sourced. We sum and report the numbers provided in their paper.

[‡]For VOLE based systems we include the communication time into the prover time for fair comparison.

Table 5: Acceleration ratio of constraint fusion and circuit squeeze.

		Prover Time (wo Cons-Opt)	Prover Time (wo Circ-Opt)	Prover Time (all Opt)
Commit advice		1.5s	0.8s	0.8s
GKR	Layer SC	6.0s	20.3s	5.7s
	Combine SC	3.3s	3.1s	3.2s
Lookup		22.4s	12.3s	12.1s
Total		33.2s	36.5s	21.8s

7.3 Effect of optimization at different levels

We also study the effect of our proposed two optimizations, constraint fusion, circuit compression, respectively.

From Table 5 we can see the comparison between our fully optimized system and our optimized system only disabling constraint and circuit optimization, respectively. The overall acceleration ratio of constraint optimization is $1.5\times$, its mainly composed of reducing the lookup procedure runtime by $1.8\times$ and reducing the commitment time on auxiliary inputs by $1.9\times$. The lookup runtime is reduced due to constraint optimization reduces the number of range relations the system has to check. The commitment time is reduced mainly because the number of committed intermediate rounding results is reduced. For circuit optimization, we can see that for the per-layer sumcheck procedure in GKR protocol, by adopting circuit optimization, we can achieve an acceleration ratio of $1.7\times$ compared with not adopting circuit optimization. Before adopting circuit optimization, the layer-wise sumcheck in GKR protocol is actually quite time consuming. After optimizing this part, the time ratio of GKR layer-wise sumcheck is reduced to 26%.

From Table 3 we can see that, compared with our system without optimization implemented in single thread, further jointly adopting constraint optimization and circuit optimization accelerates the end-to-end performance by $14.7\times$, unlocking great optimization opportunities aside from trivially accelerating everything by multi-thread parallelism.

8 Related Work

Zero-Knowledge Proof for machine learning. There has been significant research on developing ZKPs for ML. Pioneering work [68] introduced methods to prove decision trees. Subsequent efforts focused on verifying the correctness of CNN executions, as seen in [18], [37], [61] and [35]. To address inefficiencies in previous approaches, zkCNN [40] developed a specialized protocol to prove convolutions. [6] further optimized zkCNN [40].

Recently, there has been a surge in research on building ZKPs for LLMs. ZKML [10] generated proofs for GPT-2 [50] inference, requiring over an hour to generate a proof for GPT-2. Concurrently, Hao et al. [31] employed digit decomposition to prove non-linear layers in neural networks. How-

ever, their approach does not support end-to-end proof generation for LLMs. Lu et al. [41] proposed optimizations to [31], achieving significantly better performance. Another concurrent work, zkLLM [55], focused on proving the attention layer using digit decomposition. Their system is implemented purely with GPU parallelism. However, it is incompatible with provers lacking advanced GPUs.

Compared to proving inference, proving training processes is far more challenging. Several studies [1, 25, 54, 60] have explored the ZKP of training various ML models. Current approaches still involve significant overhead.

Proof aggregation. Our constraint merging technique is very different from proof-aggregation techniques like Bulletproofs [8]. They focus on reducing proof size, but cannot reduce the prover’s computation. Our constraint merging technique decreases the prover time.

9 Discussion

9.1 Analysis of different merging levels

We also explore the impact of different constraint merging levels. (A) No merging, which is the slowest case. (B) Our proposed method. (C) Fastest merging, which is conducting all beneficial mergings to the constraints. (D) Full merging, which is conducting all possible mergings to the constraints. The constraints for level (A) are introduced in Section 4.1, and the constraints for level (B) are introduced in Section 5.3. Our proposed method (B) theoretically achieves nearly ideal performance.

Next, we analyze the possible mergings that could be conducted on top of level (B).

Attention. In Equation 5, merging the computations of $q_{e_{ij}}$ and $q_{p_{ij}}$ is not feasible. This is because ensuring the correctness of $q_{p_{ij}}$ requires lookup queries that must be fully computable within the GKR circuit. Thus, all variables used that cannot be directly arithmetically computed ($q_{e_{ij}}$) has to be checked with additional lookup constraints that cannot be merged. However, merging $q_{E_{max,i}}$ and $q_{e_{i,j}}$ together is possible. The merged constraint is as follows:

$$q_{e_{ij}} = \exp(\max_j q_{E_{ij}} - q_{E_{ij}}), \text{ for } i = 1, \dots, s, j = 1, \dots, i \quad (9)$$

Level (D) adopts this merging. However, it introduces an infeasible computational cost. Specifically, checking the computation of $q_{e_{i,j}}$ using a lookup table can be modeled as a non-linear function with q_{E_i} as its input. Based on our quantization, q_{E_i} is a vector with length 32 each element is 16 bit. Consequently, the total number of possible inputs to this lookup table is $2^{16 \times 32} = 2^{512}$, resulting in a prohibitively computational cost. Both level (B) and level (C) avoid this merging since such aggressive merging significantly harms performance.

Gelu. GeLU cannot be further merged, this is due to checking Equation 6 requires checking the range relation $2q_g - 1 \leq q_x + |q_x| - \mathbb{1}_{|q_x| \geq q_{th}} \cdot |q_x| (a \cdot S_x^2 q_x^2 + b \cdot S_x |q_x| + c) < 2q_g + 1$. In the relation, $\mathbb{1}_{|q_x| \geq q_{th}}$ cannot be arithmetically computed. Thus, s has to be provided in the input layer as witness. And an additional lookup query is required to ensure the correctness of s . In conclusion, GeLU is already fully merged in level (B) and cannot be further merged.

Normalization. The layer normalization constraint for level (B) is depicted in Equation 5. Level (C) and (D) further merges q_{σ_i} with $q_{y_{ij}}$. However, it only reduces the number of rounding constraints from $sd + s$ to sd and is marginally beneficial.

9.2 Generalizability of our optimizations for matrix multiplication proof

Our efficient construction for proving matrix multiplication in Section 4.2 consists of three techniques. The first technique grouping reduces costs in general scenarios. Because grouping imposes no assumptions on the density of matrices. The second technique skipping computations based on sparsity is effective only if the padded (power of 2) matrix is sparse. However this is often the case in real-world matrix multiplications, since the width or height of the matrix often does not equal the power of 2. The third technique saving field multiplication by precomputing requires the properties of quantization to the input matrices. Thus it is mainly generalizable to most cases of zero-knowledge machine learning.

9.3 Limitations of zkGPT

The primary limitation of zkGPT's design lies in its reliance on quantization, rendering it unsuitable for directly proving the floating-point inference of neural networks. However, as previously noted, most existing ZKP systems for machine learning [10, 18, 37, 40, 41, 55] share this limitation due to their dependence on quantization. Future work that could be done includes (1) Exploring the proof generation under floating-point computation. (2) Exploring proof generation for LLM training. (3) Exploring domain-specific distributed proof generation system tailored for LLM inference/training.

10 Conclusion

In this paper, we introduce a highly efficient ZKP framework for large language models (LLMs). Our efficient methods for proving both linear and non-linear layers in LLMs offer huge performance improvements by orders of magnitude. Additionally, we propose optimization techniques at both the constraint and circuit levels. Finally, we implement our scheme on GPT-2, and the evaluation results demonstrate a significant speedup compared to existing methods.

11 Acknowledgment

We would like to extend our gratitude to the reviewers and shepherd for their constructive feedback on our work.

Ethics considerations

Our research focuses on enhancing the transparency of existing LLM cloud services. Our method allows LLM service providers to assure customers that outputs are generated correctly without compromising the model's privacy. We strictly follow ethical guidelines, avoiding deceptive practices, unauthorized disclosures, live system experiments, or any actions that could jeopardize the well-being of our team. Consequently, our work aims to improve transparency in LLM services without introducing new ethical concerns beyond those already associated with LLMs.

Open Science

We provide the source code of our study at <https://zenodo.org/records/14727819>.

References

- [1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. *Cryptology ePrint Archive*, 2024.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [3] Eleni Adamopoulou and Lefteris Moussiades. An overview of chatbot technology. In *IFIP international conference on artificial intelligence applications and innovations*, pages 373–383. Springer, 2020.
- [4] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.
- [5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [6] David Balbás, Dario Fiore, Maria Isabel González Vasco, Damien Robissout, and Claudio Soriente. Modular sum-check proofs with applications to machine learning and image processing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1437–1451, 2023.

- [7] Gautam Botrel, Thomas Piellard, Youssef El Housni, Ivo Kubjas, and Arya Tabaie. Consensys/gnark: v0.9.0, February 2023.
- [8] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.
- [9] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. Lookup arguments: improvements, extensions and applications to zero-knowledge decision trees. In *IACR International Conference on Public-Key Cryptography*, pages 337–369. Springer, 2024.
- [10] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. Zkml: An optimizing system for ml inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 560–574, 2024.
- [11] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.
- [12] Xiao Wang Chenkai Weng. Emp-zk toolkit, 2024. URL: <https://github.com/emp-toolkit/emp-zk?tab=readme-ov-file#throughput-of-circuit-based-zk-protocol>.
- [13] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. Practical verified computation with streaming interactive proofs. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pages 90–112, 2012.
- [14] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332, 2022.
- [15] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, volume 36, pages 10088–10115, 2023.
- [16] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. *Cryptology ePrint Archive*, 2022.
- [17] Jens Ernstberger, Chengru Zhang, Luca Ciprian, Philipp Jovanovic, and Sebastian Steinhorst. Zero-knowledge location privacy via accurate floating point snarks. *arXiv preprint arXiv:2404.14983*, 2024.
- [18] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. Zen: An optimizing compiler for verifiable, zero-knowledge neural network inferences. *Cryptology ePrint Archive*, 2021.
- [19] Boyuan Feng, Zheng Wang, Yuke Wang, Shu Yang, and Yufei Ding. Zeno: A type-based optimization framework for zero knowledge neural network inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 450–464, 2024.
- [20] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
- [21] OpenAI Forum. Gpt4-turbo more “stupid/lazy” - it’s not a gpt4, 2024. URL: <https://community.openai.com/t/gpt4-turbo-more-stupid-lazy-its-not-a-gpt4/608008>.
- [22] OpenAI Forum. Openai did made gpt3.5 more stupid?, 2024. URL: <https://community.openai.com/t/openai-did-made-gpt3-5-more-stupid/262979>.
- [23] Ariel Gabizon and Zachary J Williamson. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive*, 2020.
- [24] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.
- [25] Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Guru-Vamsi Policharla, and Mingyuan Wang. Experimenting with zero-knowledge proofs of training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1880–1894, 2023.
- [26] Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Yinuo Zhang. Succinct zero knowledge for floating point computations. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1203–1216, 2022.
- [27] Sergey B Gashkov and Igor S Sergeev. Complexity of computation in finite fields. *Journal of Mathematical Sciences*, 191:661–685, 2013.

- [28] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo—a turing-complete stark-friendly cpu architecture. *Cryptology ePrint Archive*, 2021.
- [29] S Goldwasser, S Micali, and C Rackoff. The knowledge complexity of interactive proof-systems. In *Proceedings of the seventeenth annual ACM symposium on Theory of computing*, pages 291–304, 1985.
- [30] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)*, 62(4):1–64, 2015.
- [31] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. Scalable zero-knowledge proofs for non-linear functions in machine learning. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3819–3836, 2024.
- [32] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [33] Bernard J Jansen and Udo Pooch. A review of web searching studies and a framework for future research. *Journal of the American Society for Information Science and Technology*, 52(3):235–246, 2001.
- [34] Fred Jelinek, Robert L Mercer, Lalit R Bahl, and James K Baker. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, 62(S1):S63–S63, 1977.
- [35] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. Scaling up trustless dnn inference with zero-knowledge proofs. *arXiv preprint arXiv:2210.08674*, 2022.
- [36] Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. I-bert: Integer-only bert quantization. In *International conference on machine learning*, pages 5506–5518. PMLR, 2021.
- [37] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. vcnn: Verifiable convolutional neural network based on zk-snarks. *Cryptology ePrint Archive*, 2020.
- [38] Alan Li, Qingkai Liang, and Mo Dong. Sparsity-aware protocol for zk-friendly ml models: Shedding lights on practical zkml. *Cryptology ePrint Archive*, 2024.
- [39] T. Liu, T. Xie, J. Zhang, D. Song, and Y. Zhang. Pianist: Scalable zkrollups via fully distributed zero-knowledge proofs. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 39–39, Los Alamitos, CA, USA, may 2024. IEEE Computer Society.
- [40] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.
- [41] Tao Lu, Haoyu Wang, Wenjie Qu, Zonghui Wang, Jinye He, Tianyang Tao, Wenzhi Chen, and Jiaheng Zhang. An efficient and extensible zero-knowledge proof framework for neural networks. *Cryptology ePrint Archive*, 2024.
- [42] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM (JACM)*, 39(4):859–868, 1992.
- [43] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [44] Alfred J Menezes, Paul C Van Oorschot, and Scott A Vanstone. *Handbook of applied cryptography*. CRC press, 2018.
- [45] Stephen Merity. The wikitext long term dependency language modeling dataset. *Salesforce Metamind*, 9, 2016.
- [46] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. Dnnfusion: accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 883–898, 2021.
- [47] NVIDIA. Nvidia a100 datasheet, 2024. URL: <https://www.nvidia.com/en-us/data-center/a100/>.
- [48] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. *arXiv preprint arXiv:1606.06031*, 2016.
- [49] Nicholas Pippenger. On the evaluation of powers and related problems. In *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*, pages 258–263. IEEE Computer Society, 1976.
- [50] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

- [51] Alex Saltanov. Openai keeps dumbing down chatgpt, 2024. URL: <https://pub.aimind.so/openai-keeps-dumbing-down-chatgpt-6a6e4a173237>.
- [52] Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980.
- [53] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.
- [54] Ali Shahin Shamsabadi, Sierra Calanda Wyllie, Nicholas Franzese, Natalie Dullerud, Sébastien Gambs, Nicolas Papernot, Xiao Wang, and Adrian Weller. Confidential-profit: confidential proof of fair training of trees. In *The Eleventh International Conference on Learning Representations*, 2022.
- [55] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. 2024.
- [56] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Annual Cryptology Conference*, pages 71–89. Springer, 2013.
- [57] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [58] Victor Vu, Srinath Setty, Andrew J Blumberg, and Michael Walfish. A hybrid architecture for interactive verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 223–237. IEEE, 2013.
- [59] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, 2018.
- [60] Suppakit Waiwitlikhit, Ion Stoica, Yi Sun, Tatsunori Hashimoto, and Daniel Kang. Trustless audits without revealing data or models. In *Forty-first International Conference on Machine Learning*, 2024.
- [61] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 501–518, 2021.
- [62] Howard Wu, Wenting Zheng, Alessandro Chiesa, Raluca Ada Popa, and Ion Stoica. DIZK: A distributed zero knowledge proof system. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 675–692, Baltimore, MD, August 2018. USENIX Association.
- [63] Min-You Wu and Daniel D Gajski. Hypertool: A programming aid for message-passing systems. *IEEE transactions on parallel and distributed systems*, 1(3):330–343, 1990.
- [64] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR, 2023.
- [65] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh, and Dawn Song. zkbridge: Trustless cross-chain bridges made practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 3003–3017, 2022.
- [66] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. Caulk: Lookup arguments in sublinear time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 3121–3134, 2022.
- [67] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Rafols. Baloo: Nearly optimal lookup arguments. *Cryptology ePrint Archive*, 2022.
- [68] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. Zero knowledge proofs for decision tree predictions and accuracy. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 2039–2053, 2020.
- [69] Jiaheng Zhang, Tianyi Liu, Weijie Wang, Yinyu Zhang, Dawn Song, Xiang Xie, and Yupeng Zhang. Doubly efficient interactive proofs for general arithmetic circuits with linear prover time. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 159–177, 2021.

A zkGPT protocol

The core part of the zkGPT protocol is integrating Lasso lookup protocol in the GKR protocol. The details are as follows: Assume our system invokes t lookup tables. For table j , we have to check the consistency of lookup vector A_j (A_j is part of the circuit’s witness and committed together with the input). We can leverage random combination to check the claim $\tilde{A}_j(r_j)$ as $\sum_{j=1}^t r_j' \tilde{A}_j(r_j) = \sum_{z \in \{0,1\}^{s_{in}}} \tilde{V}_{in}(z) \sum_{j=1}^t r_j' \tilde{C}_j'(r_j, z)$, where r_j' are random points send by the verifier. $\tilde{C}_j'(\cdot)$ returns 1 if the x -th value in A_j is the z -th value in V_{in} , otherwise returns 0. The sumcheck protocol reduces claims to $\tilde{A}_j$ to a single claim on $\tilde{V}_{in}(\cdot)$.

After invoking the Lasso protocol, we can check the consistency of lookup vectors leveraging the above method. For

Protocol 1 (zkGPT Protocol). Let C be a circuit with m layers computing an LLM with model parameters $\mathbf{W}$. Let $X_1 = \mathbf{X}$ be the input embedding, $X_m = y$ be the model's output embedding $C(\mathbf{W}, \mathbf{X})$, and X_i be the output of the i -th layer of C .

- $\text{zkGPT.KeyGen}(1^\lambda)$: Run $\text{pp} \leftarrow \text{zkPC.KeyGen}(1^\lambda)$ and outputs pp .

- $\text{zkGPT.Commit}(\mathbf{W}, \text{pp}, r)$: Define the multi-linear extension of $\mathbf{W}$ as $\tilde{\mathbf{W}}$. $\mathcal{P}$ commits to $\tilde{\mathbf{W}}$ by running $\text{com}_{\tilde{\mathbf{W}}} \leftarrow \text{zkGPT.Commit}(\tilde{\mathbf{W}}, \text{pp}, r_{\tilde{\mathbf{W}}})$ and sends $\mathcal{V}$ $\text{com}_{\tilde{\mathbf{W}}}$.

- $\langle \text{zkGPT.Prove}(\mathbf{W}, r_{\mathbf{W}}), \text{zkGPT.Verify}(\text{com}_{\tilde{\mathbf{W}}}) \rangle(\mathbf{X}, \text{pp})$:

(1) Upon receiving $\mathbf{X}$ from $\mathcal{V}$, $\mathcal{P}$ evaluates the LLM to compute the prediction $y = X_m$, the values in each layer X_i , the intermediate values aux , the lookup queries L . L is composed of $L_1 \| L_2$, where L_1 is the lookup values for the range check table, L_2 is the lookup values for the exponentiation table. $\mathcal{P}$ commits to aux, L by running $\text{com}_{\text{aux}} \leftarrow \text{zkPC.Commit}(\text{aux}, \text{pp}, r_{\text{aux}})$, $\text{com}_L \leftarrow \text{zkPC.Commit}(\tilde{L}, \text{pp}, r_L)$ and sends $\mathcal{V}$ $\text{com}_{\text{aux}}, \text{com}_L$ and $y = X_m$. Without loss of generality, we pad $\mathbf{X}, \mathbf{W}, \text{aux}, L$ to the maximum length of the four, denoted as N . We arrange the input of the circuit as $\mathbf{X} \| \mathbf{W} \| \text{aux} \| L$ of size $4N$.

(2) For the range check table and exponentiation table, $\mathcal{P}$ sends the Lasso-commitment to matrix $M_1 \in \{0, 1\}^{m_1 \times N_1}, M_2 \in \{0, 1\}^{m_2 \times N_2}$, each consisting of c different $\log(m)$ -variate multi-linear polynomials $\text{dim}_1, \dots, \text{dim}_c$, respectively. $\mathcal{V}$ picks random $r_1 \in \mathbb{F}^{\log m_1}, r_2 \in \mathbb{F}^{\log m_2}$ and sends r_1, r_2 to $\mathcal{P}$. $\mathcal{P}$ claims to $\mathcal{V}$ on $\tilde{L}_1(r_1), \tilde{L}_2(r_2)$.

(3) $\mathcal{P}$ and $\mathcal{V}$ apply Lasso Protocol on range check table and exponentiation table respectively. $\mathcal{P}$ proves that $\sum_{y \in \{0, 1\}^{\log N_1}} \tilde{M}_1(r_1, y) T_1[y] = \tilde{L}_1(r_1)$ and $\sum_{y \in \{0, 1\}^{\log N_2}} \tilde{M}_2(r_2, y) T_2[y] = \tilde{L}_2(r_2)$.

(4) $\mathcal{V}$ sends random $r^{(m)}$. Both parties compute $\tilde{X}_m(r^{(m)})$.

(5) For $i = m, m-1, \dots, 2$, with $\tilde{X}_i(r^{(i)})$,

If layer i is a matrix multiplication layer, $\mathcal{P}$ and $\mathcal{V}$ run a sumcheck protocol on $\tilde{X}_i(x, y) = \sum_z \tilde{X}_{i-1}(x, z) \cdot \tilde{W}_{i-1}(z, y)$ using the sumcheck algorithm for matrix multiplication proposed in [56]. At the end of the sumcheck, $\mathcal{V}$ receives $\tilde{X}_{i-1}(r^{(i-1)})$ and $\tilde{W}_{i-1}(r^{(i-1)})$.

If layer i belongs to other constraint layers, $\mathcal{P}$ and $\mathcal{V}$ run the GKR protocol. At the end of the sumcheck, $\mathcal{V}$ receives $\tilde{X}_{i-1}(r^{(i-1)})$ and $\text{aux}_{i-1}(r^{(i-1)})$.

(6) At the input layer, $\mathcal{V}$ has received $\tilde{X}_1(r^{(1)})$, $\tilde{W}_{i-1}(r^{(i-1)})$, $\text{aux}_{i-1}(r^{(i-1)})$ for $i = m, \dots, 1$, and $\tilde{L}_1(r_1), \tilde{L}_2(r_2)$. $\mathcal{P}$ and $\mathcal{V}$ run a sumcheck protocol to combine them into a single evaluation. At the end of the sumcheck, $\mathcal{V}$ receives $\tilde{in}(r)$.

(7) $\mathcal{V}$ validates in (r_{in}) by opening the polynomial commitments. In particular, as the size of the input is $4N$, let $r = (r_1, \dots, r_{\log N+2})$ and $r^- = (r_1, \dots, r_{\log N})$, $\mathcal{P}$ and $\mathcal{V}$ run $\langle \text{zkPC.Open}(\tilde{\mathbf{W}}, r_{\tilde{\mathbf{W}}}), \text{zkPC.Verify}(\text{com}_{\tilde{\mathbf{W}}}) \rangle(r^-, \text{pp})$ and $\langle \text{zkPC.Open}(\text{aux}, r_{\text{aux}}), \text{zkPC.Verify}(\text{com}_{\text{aux}}) \rangle(r^-, \text{pp})$. $\mathcal{V}$ receives $\tilde{\mathbf{W}}(r^-)$ and $\text{aux}(r^-)$, and evaluates $\tilde{X}_1(r^-)$ locally. $\mathcal{V}$ checks that $\tilde{in}(r) = \tilde{X}_1(r^-) \cdot (1 - r_{\log N+1}) (1 - r_{\log N+2}) + \tilde{\mathbf{W}}(r^-) \cdot (1 - r_{\log N+1}) r_{\log N+2} + \text{aux}(r^-) \cdot r_{\log N+1} (1 - r_{\log N+2}) + \tilde{L}(r^-) \cdot (1 - r_{\log N+1}) (1 - r_{\log N+2})$. If all the checks pass, $\mathcal{V}$ outputs 1; otherwise, $\mathcal{V}$ outputs 0.

the details of the entire zkGPT protocol combining Lasso and GKR protocol, please refer to Protocol 1. For the security analysis of the zkGPT protocol, please refer to Section B.

B Security Analysis of zkGPT protocol

Proof. Completeness. The completeness is straightforward by the completeness of the GKR protocol and Lasso protocol.

Soundness. For the analysis of soundness, our protocol could be viewed as the combination of GKR protocol and Lasso protocol. Suppose the prediction sent by $\mathcal{P}$ is not correctly computed, i.e., $y \neq \text{pred}(\mathbf{W}, \mathbf{X})$, but still passes the verification. This event can be divided into four cases:

- Case 1: all the values in aux, L are computed correctly. By the Schwartz-Zippel lemma [52], in Step (4) of Proto-

col 1, $\tilde{X}_m^*(r^{(m)}) \neq \tilde{X}_m(r^{(m)})$ with all but probability $\frac{1}{|\mathbb{F}|}$.

Since with high probability $\tilde{X}_m^*(r^{(m)})$ is incorrect, and in is correct, due to the soundness of GKR, the checks in Step (5) and Step (7) of Protocol 1 can only pass with negligible probability.

- Case 2: some values in aux is incorrect, L is fully computed using this aux . In this case, due to the equivalence between our defined constraints and corresponding lookup relations, there must be some values in L inconsistent with the relations in two lookup tables. Due to the soundness of Lasso protocol, the checks in Lasso can only pass with probability at most $O(\frac{m_1 + N_1^{\frac{1}{c}} + m_2 + N_2}{|\mathbb{F}|})$ which is negligible.

- Case 3: some values in aux is correct, L is inconsistent

	z-GeLU	$x\sigma(1.702x)$	h-GeLU	i-GeLU
L_2 dist	0.0054	0.012	0.031	0.0082
L_∞ dist	0.012	0.020	0.068	0.018

Table 6: Comparison of different approximation methods for GeLU. As metrics for approximation error, we report L_2 and L_∞ distance from GeLU across the range of $[-4, 4]$.

with this aux . Then some arithmetic relations checking L and aux are not satisfied. Due to the soundness of GKR, the checks in Step (5) and Step (7) of Protocol 1 can only pass with negligible probability.

- Case 4: some values in aux is incorrect, L is inconsistent with this aux . Similarly, some arithmetic relations checking L and aux are not satisfied. Due to the soundness of GKR, the checks in Step (5) and Step (7) of Protocol 1 can only pass with negligible probability.

By the union bound, the probability of $\mathcal{V}$ outputting 1 when $y \neq \text{pred}(W, X)$ is negligible.

C GeLU approximations

There have been various studies in quantization literature for approximating GeLU function. For example, [32] suggests using Sigmoid function to approximate erf: $\text{GeLU}(x) \approx x\sigma(1.702x)$, where $\sigma(\cdot)$ is the Sigmoid function. [36] proposed another approximation function for GeLU: $\text{i-GeLU}(x) = \frac{x}{2} \left[1 + \text{sgn}\left(\frac{x}{\sqrt{2}}\right) \left[a(\text{clip}(|\frac{x}{\sqrt{2}}|, \max = -b) + b)^2 + 1 \right] \right]$, where sgn denotes the sign function. Their method preserves decent accuracy and has been adopted in ZKP systems like Lu et al [41]. However, directly integrating i-GeLU into our circuit will introduce large overhead due to invoking a large number of non-arithmetic operations. This motivates the need of deriving a more ZK-friendly approximation for GeLU.

We observe that $y = \text{GeLU}(x)$ converges to $y = x$ for $x \rightarrow +\infty$, and converges to $y = 0$ for $x \rightarrow -\infty$. A natural idea occurs: we could directly use ReLU to replace GeLU when the input absolute value is large, and use a low order polynomial to handle the case when the input absolute value is small. We observe that the difference function $D(x) = \text{ReLU}(x) - \text{GeLU}(x)$ is an even function. When designing a simple function $D'(x)$ to approximate $D(x)$ on $[0, +\infty)$, we can set $D'(x) = 0$ for $x \geq t$, when $x < t$, $D'(x) = x(a \cdot x^2 + b \cdot x + c)$. Then we solve $\min_{a,b,c,t} \max_{x \in [0,t]} |D'(x) - D(x)|$

to obtain $z - \text{GeLU}$.

We compare z-GeLU along with existing approximations in Tab 6, where z-GeLU has an average error of 5.4×10^{-3} . This is $1.7\times$ more accurate than current state-of-the-art i-GeLU whose average is 8.2×10^{-2} .

Table 7: Comparison of before and after fusion.

Rounding Numbers	Normalization	Attention	GeLU
Before Fusion	$3s + 2sd$	$\frac{3}{2}hs(s+1) + sd$	$2Ds$
	49224	43584	196608
After Fusion	$s + sd$	$\frac{1}{2}hs(s+1) + sd$	Ds
	24608	30912	98304

D Efficient matrix multiplication protocol

Algorithm 1 Efficient algorithm for bookkeeping table $\tilde{A}(r||\cdot)$

- 1: **Input:** Matrix A , number of rows before padding n , number of rows before padding m , random evaluation point r , quantization bits Q (quantization range: $[0, 2^Q]$).
- 2: **Output:** array $\text{BK}[2^{\lceil \log m \rceil}]$, $\text{BK}[c] = \tilde{A}(r||c)$
- 3: Initialize array BK as zeros
- 4: Partition $r = x||y$
- 5: Compute $\tilde{e}q(x, \cdot)$, $\tilde{e}q(y, \cdot)$ using the memorization method in [58], stored in table $\text{EQ}_x[2^{\lceil \frac{\log n}{2} \rceil}]$, $\text{EQ}_y[2^{\lceil \frac{\log n}{2} \rceil}]$
- 6: Initialize array $\text{FM}[2^{\lceil \frac{\log n}{2} \rceil}][2^Q + 1]$ with zeros
- 7: Compute $\text{FM}[R][t] = \tilde{e}q(y, R) \cdot t$:
- 8: **for** $R \in \{0, 1\}^{\lceil \frac{\log n}{2} \rceil}$ **do**
- 9: **for** $t \in \{1, \dots, 2^Q\}$ **do**
- 10: $\text{FM}[R][t] = \text{FM}[R][t-1] + \text{EQ}_y[R]$
- 11: **end for**
- 12: **end for**
- 13: **for** $c \in [0, m-1]$ **do**
- 14: Initialize array $\text{AC}[2^{\lceil \frac{\log n}{2} \rceil}]$ with zeros
- 15: Compute the inner dot product:
- 16: **for** $b \in [0, n-1]$ **do**
- 17: Partition $\text{bin}(b) = b_L||b_R$
- 18: $\text{AC}[b_L] += \text{FM}[b_R][A[b][c]]$
- 19: **end for**
- 20: **for** $L \in \{0, 1\}^{\lceil \frac{\log n}{2} \rceil}$ **do**
- 21: $\text{BK}[c] += \text{EQ}_x[L] \cdot \text{AC}[L]$
- 22: **end for**
- 23: **end for**
- 24: **return** BK